Approximation Algorithms

While we have good algorithms for many optimisation problems, in the real world, the unfortunate reality is that many important practical problems turn out to be NP-hard. What can we do? Given an NP-hard problem P, we could *try* to design an efficient algorithm for it. Whether we succeed or not is another issue (if we do succeed, other than having solved P, we also get another benefit of being able to claim a million dollars from the Clay Mathematics Institute for proving $\mathbf{P} = \mathbf{NP}$). So probably that is too far-fetched. More reasonably, if we assume $\mathbf{P} \neq \mathbf{NP}$, we can't expect a polynomial time algorithm for P. What's the next best thing? Can we get a polynomial time algorithm for P that may not produce the optimal solution, but produce a solution that is *good-enough*?

Definitions

For an optimisation problem, we call an algorithm which returns a near-optimal solution, an **approximation algorithm**.

Given some optimisation problem with optimal solution value OPT, and an algorithm which produces a feasible solution with value ALG, we say that the algorithm is an approximation algorithm with an **approximation ratio** $\rho(n)$. It is also called a ρ-approximation algorithm.

For a **minimisation problem**, we define the approximation ratio as

$$\rho(n) = \max_{|I| \leq n} \frac{\text{ALG}(I)}{\text{OPT}(I)}$$

In other words, if an approximation algorithm has a approximation ratio of ρ, then for any instance I, we have

$$\frac{\text{ALG}}{\text{OPT}} \leq \rho.$$

Note that $\rho \geq 1$.

For a **maximisation problem** we define ρ to (lower) bound the same ratio $\frac{\text{ALG}}{\text{OPT}}$, so we get $0 \leq \rho \leq 1$. In some texts, a different convention is used, where the reciprocal ratio is used for defining ρ, to consistently have $\rho \geq 1$.

Ideally, for a *good* approximation algorithm, we would want our approximation ratio $\rho(n)$ to be a constant value (independent of n). This indicates, that how far the two values ALG and OPT are does not grow with the size of the input, i.e. the approximation does not become worse for larger inputs.

A general strategy to find a ρ-approximation algorithm

Suppose we are given a minimisation problem Q. Since we cannot compute OPT in polynomial time (otherwise we could decide the NP-hard decision version), we cannot compare the value of our solution against OPT directly. The usual way around this is to compare against some easy-to-compute quantity which is guaranteed to be no larger than OPT. This gives a two-step recipe.

Step 1: Find a lower bound. For instances of Q, find a quantity $L(I)$ which can be computed in polynomial time, such that

$$\mathrm{OPT}(I) \geq L(I).$$

Step 2: Convert the lower bound into a solution. Using the structure which gave us $L(I)$, construct in polynomial time a feasible solution of Q with value ALG satisfying

$$\mathrm{ALG}(I) \leq \rho \cdot L(I).$$

Putting these together,

$$\mathrm{ALG}(I) \leq \rho \cdot L(I) \leq \rho \cdot \mathrm{OPT}(I) \implies \frac{\mathrm{ALG}(I)}{\mathrm{OPT}(I)} \leq \rho,$$

so the algorithm is a ρ-approximation algorithm.

Approximating MIN-VERTEX-COVER

For the optimisation problem MIN-VERTEX-COVER, we need to find a minimum size subset of vertices, such that every edge is incident on at least one vertex in the set.

We will show a 2-approximation algorithm for MIN-VERTEX-COVER. This means, ALG $\leq$ 2 OPT, so the vertex cover produced by our algorithm will be no more than twice the size of the actual optimum.

To find a lower bound on OPT, we use a *maximal matching*. A **matching** in $G = (V, E)$ is a set of edges $M \subseteq E$ such that no two edges in M share an endpoint. A matching is **maximal** if it is not a strict subset of any other matching.[1] A maximal matching can be found in polynomial time greedily: pick any edge in the graph, add the edge to M and remove all its neighbouring edges; and repeat until the graph is empty.

Let M be a maximal matching of G. We claim that

$$\mathrm{OPT} \geq |M|.$$

Take an optimal vertex cover S^*. It must contain at least one endpoint of every edge in M. Since the edges of M share no endpoints, these endpoints are all different vertices, so S^* contains at least $|M|$ vertices. Now, to use M to get a feasible solution with value ALG, such that ALG $\leq$ 2 $|M|$, we will simply pick both endpoints of each edge in M.

Then, we can describe our approximation algorithm as follows.
1. Compute a maximal matching M of G.
2. Output C, the set of all endpoints of edges in M.

We claim that C is a vertex cover. Suppose it is not. Then there is an edge $(u, v) \in E$ with $u \notin C$ and $v \notin C$. This means that no edge of M is incident on u or v, so $M \cup \{(u, v)\}$ is also a matching, which contradicts the maximality of M. Hence every edge has an endpoint in C.

Every edge of M contributes exactly two vertices to C, and these are all distinct (since M is a matching), so $|C| = 2|M|$. Thus, we have

$$\mathrm{ALG} = |C| = 2|M| \leq 2 \cdot \mathrm{OPT}.$$

Therefore the algorithm is a 2-approximation algorithm for MIN-VERTEX-COVER.

[1] This is different from a *maximum* matching.

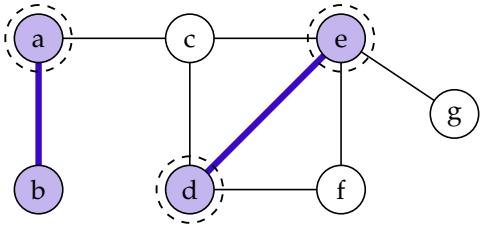


Figure 1: A run of the 2-approximation algorithm on a graph with 7 vertices. A maximal matching $M = \{(a,b),(d,e)\}$ is found, and the highlighted vertices form the output $C = \{a,b,d,e\}$, so ALG $= |C| = 4$. An optimal vertex cover is $\{a,d,e\}$, so OPT $= 3$; thus we have ALG ≤ 2 OPT.

The Traveling Salesman Problem

In the **traveling salesman problem** (TSP), we are given a complete undirected graph $G = (V,E)$ on n vertices (the "cities") with a non-negative weight $d(u,v)$ on every edge. A **tour** is a Hamiltonian cycle of G, and its weight is the sum of the weights of its edges. We want to find a tour of minimum weight. We write $w(H)$ for the total weight of a set of edges H, and OPT for the weight of an optimal tour.

The decision version of TSP is NP-complete, so we do not expect an exact polynomial-time algorithm for finding an optimal tour. Unfortunately, in the general case even approximating TSP is hard (see Problem 1). Thus we work with a special case of the problem which is more applicable in the real world.

Metric TSP

An instance of TSP is **metric** if the weights satisfy the **triangle inequality**: for all vertices i,j,k,

$$d(i,k) \leq d(i,j) + d(j,k).$$

In words, going directly from i to k is never worse than going through an intermediate city. This holds for distances in the plane, or shortest-path distances in any graph. Graphs which represent real world scenarios will usually obey the triangle inequality, so this is a reasonable special case to work with.

It turns out, even this special case is still NP-Complete. But now we will see that at least we can have constant-factor approximations for this problem.

A 2-approximation algorithm for METRIC-TSP

First we need a useful lower bound for the tour size. Let H be an optimal tour with weight OPT. Delete any one edge from this to get a Hamiltonian path in G. In fact, it is a spanning tree of G.

Let T be a minimum spanning tree of G. Then it follows, that this is a lower bound, $w(T) \leq$ OPT.

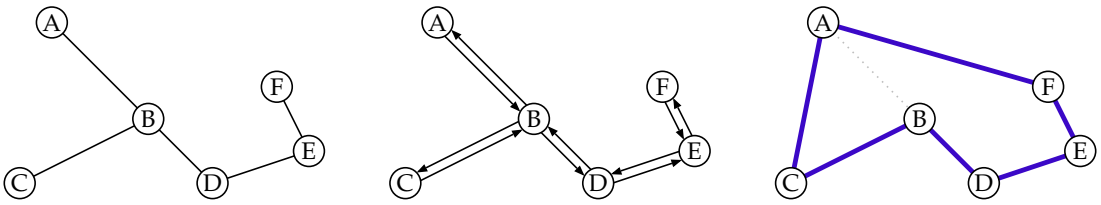
We need the algorithm to use the minimum spanning tree and construct a tour whose weight is no larger than twice the weight of the spanning tree itself.

Algorithm.

- Compute a minimum spanning tree T of G.
- Consider a Euler tour of the tree T, this is a walk W on the tree which starts and ends at the first vertex and traverses every edge in the tree twice. Clearly $w(W) = 2w(T)$.
- The walk W visits some vertices more than once. Traverse W and *shortcut* it: whenever we are at a vertex that will be visited again later, skip it and go straight to the next vertex on the

walk that is being visited for the last time. Equivalently, output the vertices of T in the order in which a depth-first traversal last visits them (a *postorder* traversal). The resulting sequence of vertices, closed up at the start, is a tour H.

Shortcutting replaces a part $x \to y_1 \to \ldots \to y_m \to z$ of the walk by the single edge (x, z). By repeated application of the triangle inequality, $d(x,z) \le d(x,y_1) + \ldots + d(y_m,z)$, so shortcutting never increases the weight. Therefore, ALG $= w(H) \le w(W) = 2w(T) \le 2$ OPT. So this is a 2-approximation algorithm for METRIC-TSP, and it runs in polynomial time.



(a) the MST T (b) the Eulerian walk W (c) the final tour H

Figure 2: The 2-approximation on an instance of metric TSP with 6 vertices.

A 1.5-approximation algorithm

In the above algorithm, we essentially doubled the edges of T to get a Euler tour of the graph. It turns out we can get a better bound by considering the even and odd degree vertices separately.

Algorithm.

1. Compute a minimum spanning tree T of G.
2. Let O be the set of vertices of odd degree in T. (The sum of degrees in any graph is even, so $|O|$ is even.)
3. Compute a **minimum weight perfect matching** M on O in the complete graph on O. This can be done in polynomial time[2].
4. In the multigraph $T \cup M$, every vertex has even degree (vertices of O gain one edge from M), and it is connected since it contains T. So it has an Eulerian cycle W.
5. Shortcut W as before to obtain a tour H.

The key claim is that the matching found has a low weight:

$$w(M) \le \frac{1}{2}\,\text{OPT}.$$

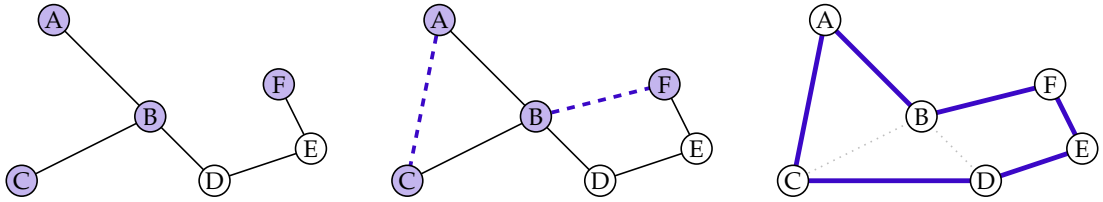
To see this, take an optimal tour of G and shortcut it so that it visits only the vertices of O (odd degree subgraph), skipping all the others. By the triangle inequality, this gives a cycle C' on O with $w(C') \le$ OPT. The cycle C' has an even number of edges, since it has $|O|$ edges, so we can split its edges into two perfect matchings M_1 and M_2 of O by taking alternate edges. Then $w(M_1) + w(M_2) = w(C') \le$ OPT, so one of them has weight at most $\frac{\text{OPT}}{2}$. Since M is a minimum weight perfect matching of O, $w(M) \le \min(w(M_1), w(M_2)) \le \frac{\text{OPT}}{2}$.

Then the Eulerian cycle W has weight $w(W) = w(T) + w(M)$, and using $w(T) \le$ OPT from before,

$$\text{ALG} = w(H) \le w(W) = w(T) + w(M) \le \text{OPT} + \frac{1}{2}\,\text{OPT} = \frac{3}{2}\,\text{OPT}.$$

[2]There exist generalisations of the Edmonds' Blossom algorithm which can compute the optimum matchings with maximum/minimum weights in polynomial time.

So this is a $\frac{3}{2}$-approximation algorithm for METRIC-TSP.



(a) T and its odd vertices (b) the matching M (dashed) (c) the tour H

Figure 3: The 1.5-approximation algorithm for Metric TSP on the same graph as in Figure 2.

Approximating MAX-E3SAT

In the MAX-E3SAT (*exactly-3 SAT*) problem, we are given a 3CNF formula ϕ with clauses $C_1, \ldots, C_m$, where every clause has exactly three literals over three distinct variables. (A clause that contains both x and $\overline{x}$ is always satisfied, so we may discard it.) We want to find an assignment of the variables which satisfies as many clauses as possible. This is a maximisation problem, and it is NP-hard, since deciding whether **all** the clauses can be satisfied is exactly E3SAT (which is NP-Complete).

In the **weighted** version, Weighted MAX-E3SAT, each clause C_j also has a weight $w_j > 0$, and we want to maximise the total weight of satisfied clauses. The unweighted problem is simply the special case with all $w_j = 1$.

A randomised $\frac{7}{8}$-approximation

The algorithm is as simple as possible: **set every variable to true or false independently with probability $\frac{1}{2}$.** Let $W = \sum_j w_j$ be the total weight of all clauses, and for each clause C_j define the indicator variable

$$Z_j = \begin{cases} 1 & \text{if } C_j \text{ is satisfied by the random assignment} \\ 0 & \text{otherwise} \end{cases}.$$

Then the weight of the clauses satisfied by the assignment is the random variable $\text{ALG} = \sum_j w_j Z_j$.

Claim. For every clause C_j, $E\left[Z_j\right] = \frac{7}{8}$.

Proof. A clause is a disjunction of three literals over three distinct variables, so it is false only if all three of its literals are false. Since the variables are set independently, each literal is false with probability $\frac{1}{2}$, independently of the others. Thus

$$\Pr\left[C_j \text{ is not satisfied}\right] = \left(\frac{1}{2}\right)^3 = \frac{1}{8},$$

and $E\left[Z_j\right] = \Pr\left[C_j \text{ is satisfied}\right] = \frac{7}{8}$. ∎

By linearity of expectation (note that Z_js are not independent),

$$E[\text{ALG}] = \sum_j w_j E\left[Z_j\right] = \frac{7}{8} W.$$

Finally, no assignment can satisfy more than all the clauses, so $\text{OPT} \leq W$. Hence

$$E[\text{ALG}] = \frac{7}{8}W \geq \frac{7}{8}\,\text{OPT}.$$

This is a **randomised $\frac{7}{8}$-approximation algorithm** for MAX-E3SAT: its output has *expected* value within a factor of $\frac{7}{8}$ of the optimum.

By virtue of being a randomised algorithm, it doesn't always guarantee the lower bound of the approximation factor, it is only in *expectation* that the value will be at least $\frac{7}{8}$ times the optimal value. As we will see in Problem 5, we can get a deterministic algorithm that always finds such an assignment by **derandomisation** of our algorithm.

Problems and Solution Sketches

1. Prove that, for every constant c, if there exists a polynomial-time c^n factor approximation algorithm for the Travelling Salesman Problem (TSP), then **P** = **NP**.

We will reduce HAMCYCLE to a c^n factor approximation for TSP to show this. In other words, if there exists a polynomial time c^n factor approximation algorithm A for TSP, then we will show that it can be used to solve HAMCYCLE in polynomial time. Since HAMCYCLE is NP-complete, this would imply **P** = **NP**.

Suppose there exists a c^n factor approximation algorithm A for TSP. This means, for an instance $G = (V, E)$ (taking $n = |V|$), the algorithm A returns a tour whose cost is no more than c^n times the cost of the optimal tour.

Let $G = (V, E)$ be an instance of HAMCYCLE, and let $n = |V|$. We will construct an instance of TSP by creating a complete graph $G' = (V, E')$ on V ($E' = \{(u, v) \mid u, v \in V, u \neq v\}$), and assign the following weights:

$$d(u,v) = \begin{cases} 1 & \text{if } (u,v) \in E \\ c^n \cdot n + 1 & \text{otherwise.} \end{cases}$$

Crucially, this construction of G' can be done in polynomial time. Notice that the value $c^n \cdot n + 1$ requires $O(n \log c + \log n)$ bits to represent, which is polynomial in the size of the input graph G.

Now, if G has a Hamiltonian cycle, then G' has the same Hamiltonian cycle, each edge having a weight of 1. Thus, the optimal tour in G' is simply this cycle itself, and it has an optimal cost of n.

If G does not have a Hamiltonian cycle, then any tour in G' must necessarily go through some edge not in E. Then, the total cost of such a tour will be at least $(c^n \cdot n + 1) + (n - 1) = c^n \cdot n + n > c^n \cdot n$

Then, apply the algorithm A on the constructed graph G'. It returns a tour with some cost ALG.
- If G contains a Hamiltonian cycle, the optimal cost is n, so A must return a tour with cost of at most ALG $= c^n \cdot n$.
- If G does not contain a Hamiltonian cycle, whatever tour it returns, its cost will be ALG $> c^n \cdot n$.

Thus, by checking whether the cost of the returned tour is at most $c^n \cdot n$, we can determine if G has a Hamiltonian cycle in polynomial time. Thus, **P** = **NP**.

2. Consider the task of scheduling n jobs into m identical machines. Each job i needs to run t_i time in any machine to complete. A job once started to run in a machine cannot be stopped midway. A schedule is an assignment $A : [n] \to [m]$. The load ℓ_j of machine j in a schedule A is $\sum_{i \in A^{-1}(j)} t_i$. The **makespan** of a schedule A is $\max_{j \in [m]} \ell_j$. Show that computing a schedule that achieves the minimum makespan is NP-complete. Design a simple 2 factor approximation algorithm for this problem. Change the algorithm suitably to improve the approximation guarantee to $\frac{3}{2}$.

Hardness:
- The decision version of the problem (MAKESPAN) is in NP, since given a schedule (assignment of jobs to machines), its makespan can be computed in polynomial time.
- We can show the reduction PARTITION $\leq_p^m$ MAKESPAN. (Recall, PARTITION asks for a partitioning of a set into two equal sum subsets.) Given numbers $a_1, \ldots, a_n$ with sum S, make n jobs with $t_i = a_i$, and let there be $m = 2$ machines. Then, an equipartition of $\{a_i\}$ exists if and only if a schedule with makespan at most $\frac{S}{2}$ exists.

Thus, MAKESPAN is NP-Complete.

2 factor approximation algorithm: Process jobs in any order, and greedily assign the current job to the machine with the smallest load currently.

Let OPT be the optimal makespan, and ALG be the makespan for the greedy scheduling. Let $T = \sum_i t_i$. First, we have the following lower bound:

$$\text{OPT} \geq \max_i t_i \geq \frac{T}{m},$$

where the first inequality is because at least one machine will have to take the longest job, and the second because the maximum cannot be less than the average.

Let the jth machine have the most load. Then, ALG $= \ell_j$. Let the last job assigned to this machine be job i. Before assigning it, machine j must have been the least loaded machine; let's call its load ℓ'_j. Then, ℓ'_j cannot exceed OPT (why?). Also, we know that OPT $\geq t_i$.

So, we have

$$\text{ALG} = \ell_j = \ell'_j + t_i \leq \text{OPT} + \text{OPT} = 2\,\text{OPT}.$$

Thus, this is a 2-factor approximation algorithm for finding the minimum makespan schedule.

1.5 factor approximation algorithm: Sort the jobs by decreasing processing time, so $t_1 \geq t_2 \geq \ldots \geq t_n$. For each job, greedily assign it to the machine which is currently least loaded.

If $n \leq m$, then every job gets its own machine, so the schedule is optimal. Otherwise, consider the $m+1$ largest jobs. By pigeonhole principle, in the optimal schedule, some machine must be assigned to at least two of these jobs. So,

$$\text{OPT} \geq 2t_{m+1}.$$

Again let j be the most loaded machine, and i be the last job scheduled to it. Then, if $i \leq m$, then by our algorithm it must be that before this scheduling, machine j was empty. Then, ALG $= t_i \leq$ OPT. Otherwise, $i \geq m+1$, so $t_i \leq t_{m+1} \leq \frac{\text{OPT}}{2}$. Then we gets

$$\text{ALG} = \ell_j = \ell'_j + t_i \leq \text{OPT} + \frac{\text{OPT}}{2} = \frac{3}{2}\,\text{OPT}$$

Thus this is a 1.5 factor approximation algorithm for finding the minimum makespan schedule.

3. Design a randomized $\frac{1}{2}$ factor approximation algorithm for the Max-Cut problem for weighted graphs. Use this to prove that, in every unweighted graph with 15 edges, there exists a cut with at least 8 cut edges.

Let $G = (V, E)$ have non-negative edge weights w_e, and let $W = \sum_e w_e$. The weight of a cut $(S, V \setminus S)$ is the total weight of the edges with exactly one endpoint in S.

Algorithm. Put each vertex into S independently with probability $\frac{1}{2}$.

Analysis. For an edge $e = (u, v)$, it is cut exactly when u and v are on different sides, which happens with probability $2 \cdot \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{2}$. By linearity of expectation, the expected weight of the cut is

$$E[\text{ALG}] = \sum_e w_e \Pr[e \text{ is cut}] = \frac{W}{2} \geq \frac{\text{OPT}}{2},$$

since no cut has weight more than W. So this is a randomised $\frac{1}{2}$-approximation algorithm.

Unweighted graphs with 15 edges. Take all weights equal to 1, so $W = 15$. The expected number of cut edges is $\frac{15}{2} = 7.5$. A random variable cannot always be below its expectation, so some cut has at least 7.5 cut edges, and since the number of cut edges is an integer, some cut has at least 8 edges.

4. Design a greedy $\frac{1}{2}$ factor deterministic approximation algorithm for the maximum cut problem on edge-weighted graphs. (Note that this is also called 2 factor approximation algorithm.)

Let $G = (V, E)$ have non-negative edge weights w_e, and let $W = \sum_e w_e$. Maintain two sets $S, \overline{S}$.

Algorithm. Order the vertices as $v_1, ..., v_n$ and start with $S = \overline{S} = \emptyset$. For $i = 1, ..., n$, place v_i as follows. Let

- a_i be the total weight of the edges between v_i and the vertices among $v_1, ..., v_{i-1}$ that are already in S, and
- b_i be the total weight of the edges between v_i and the vertices among $v_1, ..., v_{i-1}$ that are already in $\overline{S}$.

If $a_i \geq b_i$, put v_i in $\overline{S}$, so that the edges counted in a_i are cut. Otherwise, put v_i in S, so that the edges counted in b_i are cut. In both cases, the edges which get cut have total weight $\max(a_i, b_i)$.

Analysis. Every edge (v_k, v_i) with $k < i$ is counted in exactly one of a_i or b_i and in no other step, because it is only examined when its later endpoint v_i is placed, and at that moment its other endpoint is already placed on exactly one side. So the weights of all edges may be written as:

$$W = \sum_{i=1}^{n} (a_i + b_i).$$

The final cut consists exactly of those edges which were cut at the step of their later endpoint, since a placed vertex never moves, so the weight of the cut is

$$\text{ALG} = \sum_{i=1}^{n} \max(a_i, b_i) \geq \sum_{i=1}^{n} \frac{a_i + b_i}{2} = \frac{W}{2}.$$

Since no cut can contain more than the total weight, $\text{OPT} \leq W$, and hence $\text{ALG} \geq \frac{W}{2} \geq \frac{\text{OPT}}{2}$. So this is a deterministic $\frac{1}{2}$-approximation algorithm. Using appropriate data structures, this can be implemented in $O(n + m)$.

(This is just the method of derandomisation using conditional expectiation applied to the randomized algorithm from Problem 3; see box below.)

5. Design a $\frac{7}{8}$ factor deterministic approximation algorithm for the weighted Max E3SAT problem. In the E3SAT problem, every clause has exactly three distinct literals. *Hint: You can use the method of conditional expectation to derandomized the randomized $\frac{7}{8}$ factor approximation algorithm for the weighted Max E3SAT problem.*

Derandomisation using Conditional Expectation. Suppose we have a randomised algorithm A for a maximisation problem Π which uses k random bits $X_1, X_2, ..., X_k$, and we have shown that it is an α-approximation in expectation, i.e. $E[\text{ALG}(X_1, ..., X_k)] \geq \alpha \cdot \text{OPT}$. The **method of conditional expectation** is a general framework to turn A into a deterministic polynomial-time algorithm with the same guarantee.

Say $X_1 \sim \text{Ber}(p_1)$. Then

$$E[\text{ALG}(X_1, ..., X_k)] = p_1 E[\text{ALG} \mid X_1 = 1] + (1 - p_1) E[\text{ALG} \mid X_1 = 0],$$

so, since an average is at most the larger of its terms (the *averaging principle*),

$$E[\text{ALG}(X_1, ..., X_k)] \leq \max(E[\text{ALG} \mid X_1 = 1], E[\text{ALG} \mid X_1 = 0]).$$

The deterministic algorithm fixes the bits one at a time, always taking the better choice. Suppose it has already fixed $X_1 = x_1, ..., X_{i-1} = x_{i-1}$. It sets $x_i = 1$ if

$$E[\text{ALG} \mid X_1 = x_1, \ldots, X_{i-1} = x_{i-1}, X_i = 1] \geq E[\text{ALG} \mid X_1 = x_1, \ldots, X_{i-1} = x_{i-1}, X_i = 0],$$

and $x_i = 0$ otherwise. By the averaging principle applied at every step, the conditional expectation never decreases:

$$E[\text{ALG}] \leq E[\text{ALG} \mid X_1 = x_1] \leq E[\text{ALG} \mid X_1 = x_1, X_2 = x_2] \leq$$
$$\ldots \leq E[\text{ALG} \mid X_1 = x_1, \ldots, X_k = x_k] = \text{ALG}_{\text{det}}.$$

Hence $\text{ALG}_{\text{det}} \geq E[\text{ALG}] \geq \alpha \cdot \text{OPT}$: if the randomised algorithm is an α-approximation, so is the deterministic one. (For a minimisation problem, pick the *smaller* conditional expectation instead, and all the inequalities reverse.)

The only requirement is that we must be able to compute each conditional expectation $E[\text{ALG} \mid X_1 = x_1, \ldots, X_{i-1} = x_{i-1}, X_i = x_i]$ in polynomial time, which is not always straightforward.

We apply the method of conditional expectation to the randomised algorithm for weighted Max-E3SAT, which sets each variable independently to true or false with probability $\frac{1}{2}$. Here the random bits are the n variables $x_1, \ldots, x_n$, and we know $E[\text{ALG}] = \frac{7}{8}W$, where $W = \sum_j w_j$ is the total weight of all clauses.

Algorithm. For $i = 1, \ldots, n$, having fixed $x_1 = b_1, \ldots, x_{i-1} = b_{i-1}$, compute

$$E[\text{ALG} \mid x_1 = b_1, \ldots, x_{i-1} = b_{i-1}, x_i = 1] \quad \text{and} \quad E[\text{ALG} \mid x_1 = b_1, \ldots, x_{i-1} = b_{i-1}, x_i = 0],$$

and set b_i to the value with the larger conditional expectation.

Computing the conditional expectations. Fix a partial assignment, and let the remaining variables be uniformly random. By linearity of expectation,

$$E[\text{ALG} \mid \text{partial assignment}] = \sum_j w_j \Pr\big[C_j \text{ is satisfied} \mid \text{partial assignment}\big].$$

For a clause C_j, if one of its literals is already true then this probability is 1. Otherwise, the clause is satisfied unless all of its remaining literals are false. If r of its literals have variables which are still unset (the others being false), each of these is false independently with probability $\frac{1}{2}$, so the probability is $1 - 2^{-r}$. So the conditional expectation can be computed in $O(m)$ time (by finding m probabilities for the m clauses).

Clearly, the conditional expectation never decreases along the algorithm, so the final assignment has weight

$$\text{ALG}_{\text{det}} \geq E[\text{ALG}] = \frac{7}{8}W \geq \frac{7}{8}\,\text{OPT},$$

since $\text{OPT} \leq W$. The algorithm makes $2n$ evaluations of $O(m)$ time each, so it runs in $O(nm)$ time, and is a deterministic $\frac{7}{8}$-approximation algorithm.

6. Show that every E3SAT problem instance with at most 7 clauses is always satisfiable.

Let ϕ be an E3SAT formula with $m \leq 7$ clauses, and choose an assignment uniformly at random. Each clause has three distinct variables, so it is unsatisfied with probability $\frac{1}{8}$. By linearity of expectation, the expected number of unsatisfied clauses is $\frac{m}{8} \leq \frac{7}{8} < 1$. A non-negative integer valued random variable with expectation less than 1 must take the value 0 with positive probability, so some assignment leaves 0 clauses unsatisfied. Hence ϕ is satisfiable.